Mykyta Nikitenko

(pronounced /mɪ'kiːtə nɪkɪ'tɛnkoʊ/)

Rust / AI Systems Engineer · Agent Infrastructure · Distributed Systems · 10+ years

Pronouns: He/His

📍 Kraków, Poland | Remote (GMT+1 to PST-8)
✉️ nikitanikitenko@gmail.com
🔗 LinkedIn | GitHub
Languages: English (C1), Polish (C1), Ukrainian/Russian (Native)

Profile

Systems engineer with 10+ years building distributed systems, storage, and ML pipelines in production, and now infrastructure for coding agents.

I am looking for a role where Rust is the primary/target language and where I own systems. For me Rust has become a primary instrument in the era of LLM-augmented engineering, with a type system defining constraints that allow producing reliable code on production scale with faster feedback loops.

I prefer to write clean simple code following best community practices and using pedantic Clippy. I always think in advance about how a system is going to behave under edge cases and how to avoid them. I design software with the ability to reproduce or debug any issue. I know how to adapt an existing codebase or create a new one based on requirements. I use my machine learning and data science knowledge to create an optimal solution under defined constraints. In my mind, I imagine systems in parts and as a whole as a mathematical deterministic system.

As a hobby I play piano, sing in the Krakofonia choir with my wife, happy father of 5 cats and 2 dogs. I appreciate direct, honest communication and working with teams that respect and support each other.

Open Source

rudzio

async test framework for Rust

crates.io | GitHub | docs.rs

I have discovered that long-running agent-driven work depends on test design, and that using `cargo test` in monolithic codebases is frustrating. Rudzio runs tests from multiple crates in a single process with non-blocking interactive terminal output and each runtime on its own thread, handling panics, SIGTERM/SIGINT, suite setup errors, timeouts, cancellation and teardown. It organises tests into Suites, which might define complex setup and teardown logic guaranteed to be called. Resource ownership is a lifetime hierarchy enforced by the compiler: `'runtime`: `'suite_context`: `'test_context`. This allows writing tests that, for example, create a Podman container, always perform cleanup even when a panic appears, create a connection pool, borrow a connection per test. It is possible to plug in any asynchronous runtime and Rudzio is completely runtime agnostic (currently supports tokio, compio, async-std, smol, monoio, embassy and futures-executor).

Technologies: 🦀 Rust, proc macros, cargo internals, async runtimes, threading, panic unwinding, cancellation.

Projects

PorfiryPetrovichAI

coding-agent infrastructure · my daily engineering tool · private, walkthrough on request

A self-hosted agent system built since April 2026, largely by agents running on the system itself. Built on hard forks of opencode and maki (Rust), with ~380 own commits across the agent core, model router, context pruning and memory plugins, kept as commits on the vendored trees with no patch files.

- Parallel subagents, each in its own git worktree and branch, with serialized merge-back and git as the source of truth for task results; an inter-agent control channel (SQLite) for steering running agents; self-hosted persistent memory per repository.

- LLM gateway (LiteLLM): provider-neutral model names, usage-based routing, and fallback chains ending in a local model (Ollama); GPU-backed context-compression service (ModernBERT).
- Infrastructure: Fedora CoreOS on Hetzner via OpenTofu and GitHub Actions, rpm-ostree image bake, Podman quadlets, Nix, observability with SigNoz and OpenTelemetry.

Technologies: TypeScript, Bun, 🦀 Rust, Lua, Julia, 🐍 Python, Nix, OpenTofu, Podman, SQLite

Fugue

Memory-safe systems programming language guaranteed by linear types, without GC or a panic system, with an optional completion-based I/O thread-per-core async runtime. Compiler prototype in progress, private.

Professional Experience

Senior Rust Engineer at Fathom.io (Remote) October 2025 - Now

Redesigned the file storage engine to support multiple swappable generic backends (GCP, S3, Minio) with stronger consistency guarantees and failover. It uses async streams to avoid materialising data in memory, and supports resumable transfers, range requests, uploads and downloads over a generic thin API. Built with a focus on observability and testability: everything can be validated by unit, integration and end-to-end tests based on requirements. It became the foundation for a new asynchronous query execution layer, which stores intermediate results in storage and reads them back as a stream through a thin asynchronous failover API.

Technologies: 🦀 Rust, ☁️ S3, ☁️ Google Cloud Storage, 🔭 OpenTelemetry

---

Senior Software Engineer at NXLog Platform (Remote) May 2025 - October 2025

Shipped a functional prototype of a stateless Rust service replacing a legacy Java system for managing a fleet of NXLog data-collection agents, with one binary covering on-premise, cloud and multitenant installations. I designed the multitenancy model. For constrained on-premise setup, I designed async communication over Postgres replication protocol, swappable per deployment strategy with SQS or RabbitMQ.

Technologies: 🦀 Rust, 🐘 PostgreSQL, ☸️ Kubernetes, 🐋 Docker, ☁️ GCP

Senior Software Engineer at daily.dev (Remote) June 2023 - May 2025

I designed a new machine learning pipeline for a deep learning recommendation engine serving over half a million developers at around 100 ms, with asynchronous near real-time inference. It enabled continuous learning with feedback loops, creating experiments at runtime, and adapting model hyperparameters for specific users based on their interactions with the platform. It replaced a legacy pipeline based on collecting trivial statistics, and gave full observability, A/B testing, the ability to reproduce any experiment, and full coverage of unit, integration and end-to-end tests.

A/B tested against the previous pipeline: +5% upvotes, +44% bookmarked posts, +13% significant events, +11% unique posts clicked.

daily.dev engineering blog: [Project Sauron: building a two-tower retrieval model for personalized recommendations at daily.dev]

Technologies: 🦀 Rust, 🐹 Go, 🐍 Python, 🗄️ ClickHouse, 🗄️ Redis, ☁️ GCP

Senior Software Engineer at OLX (Real Estates) (Remote) June 2020 to May 2023

OLX Real Estate platforms (otodom.pl, imovirtual.pt and storia.ro) faced scalability and maintainability issues with a legacy 1.5 million line PHP monolith, a database of 300+ MySQL tables, and outdated infrastructure. I established engineering principles for splitting the monolith into Go microservices and migrating to AWS, following best practices for modern cloud applications with a focus on observability, testability and scalability. This allowed a complete redesign of the agent admin UI and listings management based on user engagement data, listing statistics and ML predictions for advertisement bump timing, releasing the agent UI early after internal testing with selected clients.

Technologies: 🐹 Go, ☁️ AWS

Senior Software Engineer at Luxoft for NAUTO.com (In-Office) May 2019 - May 2020

I designed and created a new incident detection (dangerous behaviour or items) and driver identification production pipeline in Go from a Jupyter Python prototype, for a continuous stream of events collected by safety cameras inside commercial fleets, which allowed new deep learning algorithms to be deployed with a focus on determinism and testability.

Scoped to specific safety events instead of continuous behavioural profiling.
Presentation at San Francisco ScyllaDB Conference 2019 – "Nauto on Consistently Merging Time Series and Finding Similar Images"
Technologies: 🐹 Go, 📡 Kafka, ☁️ AWS, 🗄️ Scylla

Earlier

Full-Stack Software Engineer, Andersen (In-Office) · June 2018 to May 2019
Built internal banking platform with microservices architecture for business process modeling and data computation. Technologies: Go, Angular

Full Stack Software Engineer, Statsminder Startup at International Telecoms (Hybrid) · January 2017 to May 2018
Built VoIP platform handling millions of concurrent SIP connections for international route optimization. Technologies: Go, Angular, FreeSwitch

Student internships, Full-Stack Software Engineer, Multimedia Nordic and Gillie Software (In-Office) · January 2016 to December 2016
CRM for an oil industry client in Norway; web apps with digital maps and real-time call center analytics. Technologies: .NET, AngularJS

Skills

🦀 **Rust:** Using Rust since 2017, full-time since 2023, building cloud services, ML pipelines, and distributed systems. Memory safety: lifetimes and ownership, unsafe and FFI. Type system: traits, static and dynamic dispatch, type-safe patterns, GATs, HRTB, zero-cost abstractions. Async: runtimes (tokio, compio), hyper, axum, futures, streams and cancellation, allocator API. API design: trait-based abstractions, idiomatic error handling. Tooling: proc macros, compile-time guarantees, pedantic Clippy. Database drivers, HTTP services, optimizations and performance profiling.

🤖 **AI / Agent Infrastructure:** LLM gateways (LiteLLM): routing, provider fallback, local models via Ollama. Coding-agent harnesses: parallel subagents in isolated git worktrees, context pruning and compression, persistent memory, inter-agent messaging. Production ML pipelines: recommendation and deep learning inference.

🗄️ **Distributed Systems & Storage:** PostgreSQL, Clickhouse, Scylla, Redis: relational and non-relational data modeling. Object storage: S3, GCS, Minio. RabbitMQ, Kafka, SQS, SNS, Google Pub/Sub; CDC, Debezium. Designing event-driven systems.

🧪 **Testing, Reliability & Observability:** TDD, BDD. Unit, integration and e2e tests as distinct tools: deterministic tests, and any defect reproducible by a test. Tracing and observability.

💻 **Linux & Containerization:** processes, threads, signals, scheduling; file descriptors, pipes, sockets, io-uring; syscalls and the syscall ABI (x86-64); namespaces. Fedora/RHEL ecosystem. Nix and NixOS: declarative system configuration, reproducible environments. Daily user since 2009. Docker, Podman, Kubernetes, DevContainers, Toolbox, Distrobox, rpm-ostree.

Other languages: Go (Advanced), Python (Advanced), JavaScript/TypeScript (Advanced)

⚙️ **Paradigms:** Object-oriented, functional, concurrent, zero-copy; *idiomatic* Rust and Go.

🔧 **Revision Control:** Git. Git workflows. Branching, merging, rebasing, bisecting, cherry-pick, reflog. Submodules and worktrees. Merge queues. Designing Git workflows for LLM tooling and throughput.

Education

Machine Learning, Data Science and Deep Learning
MasterCode IT School, 2018-19

System Engineering
Kharkiv Polytechnic Institute, Faculty of Automation and Device Engineering, 2014-17